

Feladatleírás:

Projekt: Agentic RAG Chatbot Prototípus Fejlesztése

A feladat egy Agentic RAG (Retrieval-Augmented Generation) alapú chatbot prototípus fejlesztése Pythonban, a LangGraph framework segítségével. A cél egy működőképes, jól dokumentált és reprodukálható megoldás, amely bizonyítja az agentic rendszerek tervezésének és a moduláris RAG alrendszerek integrációjának magabiztos ismeretét.

Problémameghatározás és Adatforrás

Problémaválasztás: Válassz egy valós problémát (domain/use case), amelyre a chatbot megoldást nyújt.

Indoklás: A dokumentációban röviden indokold a választásodat a következő szempontok szerint:

- Miért releváns a probléma?
- Milyen felhasználói igényt elégít ki?
- Miért előnyös rá az agentic RAG megközelítés? **Architektúra és Agentic**

Működés

Agentic Workflow (LangGraph): Készíts egy agentic workflow-t a LangGraph frameworkkel, amely legalább 5 node-ot tartalmaz. A gráfnak tartalmaznia kell:

- Autonóm döntéshozatalt (pl. conditional routing).
- Részfeladatokra bontást és önálló végrehajtást.
- Állapotkezelést a köztes eredmények tárolására.

Eszközök (Tools): Integrálj legalább 2 eszközt (tool) a workflow-ba. A RAG funkcionalitás mellett legyen legalább egy, nem pusztán visszakeresési célú eszköz is.

RAG Alrendszer: Hozz létre egy dedikált, moduláris RAG algráfot (subgraph), amely a fő workflow-ból hívható, de nem számít bele a 3-5 csomópontba.

Adatforrás: Használj szabadon választott szöveges adatforrást (pl. PDF dokumentumok, nyilvános adathalmazok, cikkek). A hangsúly a minőségi feldolgozáson és a skálázható adatintegráción van, nem a mennyiségen.

Technikai Megvalósítás és UI

Modellválasztás: Ne használj fizetős API-kat. Válassz egy a helyi erőforrásaidhoz illeszkedő, nyílt forráskódú LLM-et, és a dokumentációban röviden indokold a választásod (trade-offok). Amennyiben ez nem lehetséges, dummy LLM-ek használata is elfogadott.

Felhasználói Felület (UI): Készíts egy egyszerűsített prototípus UI-t Streamlit használatával, amely bemutatja az ágens működésének főbb lépéseit és a RAG folyamat eredményét.

Futtatási Környezet: A megoldásod legyen konténerizált. A Dockerfile elkészítése kötelező. Előny, ha a több komponenst (pl. UI, API) igénylő megoldásokat dockercompose.yml-lel fogod össze.

Értékelés és Teljesítményelemzés

Funkcionális Értékelés: Állíts össze egy 10–20 kérdésből álló mini értékelő készletet a választott problémához és értékeld ki a rendszered teljesítmény akár 1 node-ra vagy a teljes agentic workflow-ra.

Teljesítményteszt (Load Scenario): Mutass be egy egyszerűsített terheléses tesztet (50– 200 lekérdezés). Az eredményeket foglald össze:

- Alapvető latency metrikák.
- A rendszer fő szűk keresztmetszetének (bottleneck) azonosítása.
- 1-2 konkrét optimalizálási javaslat.

Leadandók és Értékelési Szempontok

Leadandók:

- A teljes projekt forráskódja egy Git repozitóriumban.
- Dockerfile (és opcionálisan docker-compose.yml) a reprodukálhatósághoz.
- Egy README.md dokumentáció, amely tartalmazza:
 - A probléma és a célkitűzés leírását.
 - A rendszerarchitektúra áttekintését és a tervezési döntések indoklását.
 - A funkcionális értékelés és a teljesítményteszt eredményeinek összefoglalását.
 - Telepítési és futtatási útmutatót.

Értékelési szempontok:

- Kódminőség és olvashatóság.
- A megoldás reprodukálhatósága.

- A problémaválasztás relevanciája és indoklása.
- Az agentic architektúra és a LangGraph implementáció minősége.
- Az értékelési módszertan és a levont következtetések.
- A teljesítményelemzés és a bottleneck analízis mélysége.